ARYAN SANDAL 2024CSB1103
MOINAK GOSWAMI 2024CSB1225

Trace-Driven Memory Cache Simulator

Phases 1, 2 & 3 Implementation

Phase 1: Environment Setup & Trace Generation

Technical Write-up

Dynamic binary instrumentation via the Intel PIN tool operates by utilizing a Just-In-Time (JIT) compiler to inject custom analysis routines into the application's instruction stream at runtime. For this phase, the tool intercepts memory read and write operands before they are executed by the CPU.

The instrumentation script identifies whether an instruction performs a memory read or write using `INS_MemoryOperandIsRead` and `INS_MemoryOperandIsWritten`. When detected, it inserts a predicated call to our analysis functions (`RecordMemRead` or `RecordMemWrite`), capturing the Instruction Pointer (IP) and the Effective Address (EA). This generates the raw memory access trace necessary for the simulator. The overhead is minimized by writing directly to a buffered file stream.

PIN Tool Implementation

```
#include "pin.H"
#include <iostream>
#include <fstream>

FILE* trace;

VOID RecordMemRead(VOID* ip, VOID* addr) {
    fprintf(trace, "R %p %p\n", ip, addr);
}

VOID RecordMemWrite(VOID* ip, VOID* addr) {
    fprintf(trace, "W %p %p\n", ip, addr);
}

VOID Instruction(INS ins, VOID* v) {
    UINT32 memOperands = INS_MemoryOperandCount(ins);

    for (UINT32 memOp = 0; memOp < memOperands; memOp++) {
        if (INS_MemoryOperandIsRead(ins, memOp)) {
            INS_InsertPredicatedCall(
                ins, IPOINT_BEFORE, (AFUNPTR)RecordMemRead,
                IARG_INST_PTR,
```

```
                IARG_MEMORYOP_EA, memOp,
                IARG_END);
        }
        if (INS_MemoryOperandIsWritten(ins, memOp)) {
            INS_InsertPredicatedCall(
                ins, IPOINT_BEFORE, (AFUNPTR)RecordMemWrite,
                IARG_INST_PTR,
                IARG_MEMORYOP_EA, memOp,
                IARG_END);
        }
    }
}

VOID Fini(INT32 code, VOID* v) {
    fclose(trace);
}

int main(int argc, char* argv[]) {
    if (PIN_Init(argc, argv)) return -1;

    trace = fopen("pinatrace.out", "w");

    INS_AddInstrumentFunction(Instruction, 0);
    PIN_AddFiniFunction(Fini, 0);

    PIN_StartProgram();

    return 0;
}
```

Phase 2 & 3: Base Simulator Development & Optimal Policies

Technical Write-up

The core cache simulator models a configurable N-way set-associative architecture. Address translation is performed using bitwise shift and mask operations rather than floating-point division to ensure low-latency processing of multi-gigabyte traces.

Given a 64-bit memory address, the extraction functions isolate the Tag, Index, and Block Offset. The bit lengths are calculated during cache initialization:

$$\text{OffsetBits} = \log_2(\text{BlockSize})$$
$$\text{IndexBits} = \log_2\left(\frac{\text{CacheSize}}{\text{BlockSize} \times \text{Associativity}}\right)$$

The system implements LRU (Least Recently Used), LFU (Least Frequently Used), Random, and Belady's MIN algorithm.

- **LRU & LFU:** Evaluated using dynamic temporal timestamps and access frequency counters at the `CacheLine` level. LFU breaks ties by falling back to LRU logic to prevent cache pollution from stale, high-frequency blocks.
- **Belady's MIN (Optimal Baseline):** Requires omniscient knowledge of the workload. This is achieved via a two-pass mechanism. The simulator's `preprocessTrace` function performs an initial pass over the trace, mapping each memory tag to a `std::queue`-like structure (implemented as a dynamic vector of indices) representing its future accesses. During the simulation pass, if an eviction is necessary, the algorithm iterates through the set and evicts the block whose next access index is the furthest in the future, establishing the theoretical upper bound for the hit rate.

Simulator Engine Implementation

```
#include <bits/stdc++.h>
using namespace std;
enum Policy
{
    LRU,
    LFU,
    RANDOM,
    OPTIMAL
};

struct CacheLine
{
    bool f;
    bool dirty;
    uint64_t tag;
    uint64_t lastUse;
    uint64_t freq;

```

```
    CacheLine() : f(false), dirty(false), tag(0), lastUse(0), freq(0) {}
};

class Cache
{
    uint32_t num_sets;
    uint32_t associativity;
    uint32_t block_size;
    Policy policy;
    uint32_t idxBits;
    uint32_t offset_bits;
    vector<vector<CacheLine>> sets;
    uint64_t currTime;
    unordered_map<uint64_t, vector<uint64_t>> future_accesses;
    uint64_t global_access_index;

public:
    uint64_t hits, misses, evictions;

    Cache(uint32_t size, uint32_t assoc, uint32_t blk_size, Policy pol)
        : associativity(assoc), block_size(blk_size), policy(pol), currTime(0), global_access_index(0), hits(0), misses(0), evictions(0)
    {

        num_sets = size / (block_size * associativity);
        idxBits = log2(num_sets);
        offset_bits = log2(block_size);
        sets.resize(num_sets, vector<CacheLine>(associativity));
    }

    void preprocessTrace(const vector<uint64_t> &trace)
    {
        for (size_t i = 0; i < trace.size(); ++i)
        {
            uint64_t tag = trace[i] >> (offset_bits + idxBits);
            future_accesses[tag].push_back(i);
        }
    }

    void access(uint64_t addr, char type)
    {
        currTime++;
        global_access_index++;

        uint64_t index = (addr >> offset_bits) & ((1 << idxBits) - 1);
        uint64_t tag = addr >> (offset_bits + idxBits);

        auto &set = sets[index];
        int hitIdx = -1, emptyIdx = -1;

        for (uint32_t i = 0; i < associativity; ++i)
        {
            if (set[i].f)
            {
                if (set[i].tag == tag)
                {
```

```
                    hitIdx = i;
                    break;
                }
            }
            else if (emptyIdx == -1)
            {
                emptyIdx = i;
            }
        }

        if (hitIdx != -1)
        {
            hits++;
            set[hitIdx].lastUse = currTime;
            set[hitIdx].freq++;
            if (type == 'W')
                set[hitIdx].dirty = true;
        }
        else
        {
            misses++;
            int replace_idx = emptyIdx;

            if (replace_idx == -1)
            {
                evictions++;
                replace_idx = findVictim(set, tag);
            }

            set[replace_idx].f = true;
            set[replace_idx].tag = tag;
            set[replace_idx].lastUse = currTime;
            set[replace_idx].freq = 1;
            set[replace_idx].dirty = (type == 'W');
        }

        if (policy == OPTIMAL)
        {
            auto &accesses = future_accesses[tag];
            if (!accesses.empty() && accesses.front() <=
   global_access_index)
            {
                accesses.erase(accesses.begin());
            }
        }
    }

private:
    int findVictim(const vector<CacheLine> &set, uint64_t current_tag)
    {
        if (policy == RANDOM)
        {
            static mt19937 gen(1337);
            uniform_int_distribution<> dis(0, associativity - 1);
            return dis(gen);
        }

        int victim_idx = 0;
```

```
131
132         if (policy == LRU)
133         {
134             uint64_t miniTime = set[0].lastUse;
135             for (uint32_t i = 1; i < associativity; ++i)
136             {
137                 if (set[i].lastUse < miniTime)
138                 {
139                     miniTime = set[i].lastUse;
140                     victim_idx = i;
141                 }
142             }
143         }
144         else if (policy == LFU)
145         {
146             uint64_t min_freq = set[0].freq;
147             uint64_t miniTime = set[0].lastUse;
148             for (uint32_t i = 1; i < associativity; ++i)
149             {
150                 if (set[i].freq < min_freq ||
151                     (set[i].freq == min_freq && set[i].lastUse <
    miniTime))
152                 {
153                     min_freq = set[i].freq;
154                     miniTime = set[i].lastUse;
155                     victim_idx = i;
156                 }
157             }
158         }
159         else if (policy == OPTIMAL)
160         {
161             uint64_t max_future_dist = 0;
162             for (uint32_t i = 0; i < associativity; ++i)
163             {
164                 uint64_t tag = set[i].tag;
165                 auto &accesses = future_accesses[tag];
166                 uint64_t next_use = (accesses.empty()) ? UINT64_MAX :
    accesses.front();
167
168                 if (next_use == UINT64_MAX)
169                 {
170                     return i;
171                 }
172                 if (next_use > max_future_dist)
173                 {
174                     max_future_dist = next_use;
175                     victim_idx = i;
176                 }
177             }
178         }
179         return victim_idx;
180     }
181 };
182
183 int32_t main()
184 {
185     uint32_t cache_size = 32768;
186     uint32_t block_size = 64;
```

```
    uint32_t associativity = 8;
    Policy target_policy = LRU;

    Cache sim_cache(cache_size, associativity, block_size,
   target_policy);

    ifstream trace_file("pinatrace.out");
    if (!trace_file.is_open())
        return 1;

    char type;
    string ip;
    uint64_t addr;

    vector<pair<char, uint64_t>> trace_data;
    vector<uint64_t> addr_only_trace;

    while (trace_file >> type >> ip >> hex >> addr)
    {
        trace_data.push_back({type, addr});
        if (target_policy == OPTIMAL)
        {
            addr_only_trace.push_back(addr);
        }
    }

    if (target_policy == OPTIMAL)
        sim_cache.preprocessTrace(addr_only_trace);

    for (const auto &access : trace_data)
    {
        sim_cache.access(access.second, access.first);
    }

    cout << "Total Accesses: " << sim_cache.hits + sim_cache.misses <<
   "\n";
    cout << "Hits: " << sim_cache.hits << "\n";
    cout << "Misses: " << sim_cache.misses << "\n";
    cout << "Evictions: " << sim_cache.evictions << "\n";

    double hit_rate = (double)sim_cache.hits / (sim_cache.hits +
   sim_cache.misses) * 100.0;
    cout << "Hit Rate: " << hit_rate << "%\n";

    return 0;
}
```